Week 4, Notebook 1: Graph Neural Networks from Scratch

Pure Python + NumPy — Understanding Message Passing

What you'll build: A Graph Neural Network from absolute zero — nodes, edges, message passing, and node classification.

New concepts:

- Graphs as data structures for neural networks
- Adjacency matrices and degree matrices
- Message passing: aggregate neighbors, update node features
- Graph Convolutional Network (GCN) layer math

Builds on: Week 1–2 fundamentals (forward pass, backprop, activations)

Time estimate: 60 minutes

Why Graphs?

Most real data isn't a grid (images) or a sequence (text). Social networks, molecules, road maps, knowledge graphs — these are all **graphs**. GNNs learn representations by passing messages between connected nodes.

Part 1: Graphs as Matrices

In [1]

```python
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)

# ============================================================
# A graph is just: nodes + edges
# We represent it as an adjacency matrix A
# ============================================================

# Example: Zachary's Karate Club (simplified to 8 nodes, 2 classes)
# Each node is a club member; edges = friendships
# Task: predict which faction (0 or 1) each member belongs to

# Adjacency matrix: A[i,j] = 1 if nodes i and j are connected
A = np.array([
    [0, 1, 1, 1, 0, 0, 0, 0],  # Node 0
    [1, 0, 1, 0, 0, 0, 0, 0],  # Node 1
    [1, 1, 0, 1, 1, 0, 0, 0],  # Node 2
    [1, 0, 1, 0, 0, 0, 0, 0],  # Node 3
    [0, 0, 1, 0, 0, 1, 1, 0],  # Node 4
    [0, 0, 0, 0, 1, 0, 1, 1],  # Node 5
    [0, 0, 0, 0, 1, 1, 0, 1],  # Node 6
    [0, 0, 0, 0, 0, 1, 1, 0],  # Node 7
], dtype=float)

# Node features: each node has a 3D feature vector
# (In real tasks: user profile info, atom properties, etc.)
X = np.random.randn(8, 3) * 0.5
X[:4] += np.array([1.0, 0.0, 0.5])   # Faction 0 bias
X[4:] += np.array([-1.0, 0.5, -0.5]) # Faction 1 bias

# Labels
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])

print(f"Graph: {A.shape[0]} nodes, {int(A.sum() / 2)} edges")
print(f"Node features shape: {X.shape}")
print(f"Labels: {y}")

# Degree matrix: D[i,i] = number of neighbors of node i
D = np.diag(A.sum(axis=1))
print(f"\nDegree matrix diagonal: {np.diag(D)}")

# Visualize the graph
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Graph structure
pos = {
    0: (0, 1), 1: (1, 2), 2: (1, 0.5), 3: (0, -0.5),
    4: (3, 0.5), 5: (4, 2), 6: (4, 0), 7: (5, 1),
}
for i in range(8):
    for j in range(i+1, 8):
        if A[i, j] == 1:
            xi, yi = pos[i]
            xj, yj = pos[j]
            axes[0].plot([xi, xj], [yi, yj], 'gray', linewidth=1, alpha=0.5)

for i in range(8):
```

```
    color = 'steelblue' if y[i] == 0 else 'coral'
    xi, yi = pos[i]
    axes[0].scatter(xi, yi, c=color, s=400, zorder=5, edgecolors='black', linewidth=1.5)
    axes[0].text(xi, yi, str(i), ha='center', va='center', fontsize=12, fontweight='bold', color='white')

axes[0].set_title('Graph Structure (colored by class)')
axes[0].set_aspect('equal')
axes[0].axis('off')

# Adjacency matrix
im = axes[1].imshow(A, cmap='Blues', vmin=0, vmax=1)
axes[1].set_title('Adjacency Matrix A')
axes[1].set_xlabel('Node')
axes[1].set_ylabel('Node')
for i in range(8):
    for j in range(8):
        axes[1].text(j, i, int(A[i,j]), ha='center', va='center', fontsize=10)

plt.tight_layout()
plt.savefig('w4_01_graph.png', dpi=100, bbox_inches='tight')
plt.show()
```

Part 2: The GCN Layer — Message Passing in Matrix Form

The core GCN operation is beautifully simple:

H' = σ(D■■⁻¹/² Ã D■■⁻¹/² H W)

Where:

- **Ã = A + I** (adjacency + self-loops: every node also "messages" itself)
- **D■** = degree matrix of Ã
- **H** = node features (N × F)
- **W** = learnable weight matrix (F × F')
- σ = activation (ReLU)

In plain English: **each node averages its neighbors' features (including itself), then transforms through a weight matrix.**

In [2]

```python
# ============================================================
# GCN Layer from scratch
# ============================================================

class GCNLayer:
    """A single Graph Convolutional Network layer."""

    def __init__(self, in_features, out_features):
        # He initialization
        self.W = np.random.randn(in_features, out_features) * np.sqrt(2.0 / in_features)
        self.b = np.zeros(out_features)

        # Gradients
        self.dW = None
        self.db = None

        # Cache
        self.H_in = None
        self.Z = None
        self.A_hat = None

    def forward(self, H, A_hat):
        """
        H: node features (N x F_in)
        A_hat: normalized adjacency (N x N)
        Returns: activated output (N x F_out)
        """
        self.H_in = H
        self.A_hat = A_hat

        # Message passing: aggregate neighbor features
        # AH = A_hat @ H  (each node gets weighted sum of neighbor features)
        self.AH = A_hat @ H

        # Transform: apply learnable weights
        self.Z = self.AH @ self.W + self.b

        # Activate
        self.out = np.maximum(0, self.Z)  # ReLU
        return self.out

    def backward(self, d_out):
        """Backprop through the GCN layer."""
        # Through ReLU
        dZ = d_out * (self.Z > 0).astype(float)

        # Gradients for W and b
        m = self.H_in.shape[0]
        self.dW = (self.AH.T @ dZ) / m
        self.db = np.mean(dZ, axis=0)

        # Gradient for input (to pass to previous layer)
        dAH = dZ @ self.W.T
        dH = self.A_hat.T @ dAH  # Back through message passing
        return dH


def normalize_adjacency(A):
```

```
    """Compute D_hat^{-1/2} A_hat D_hat^{-1/2} (symmetric normalization)."""
    A_hat = A + np.eye(A.shape[0])  # Add self-loops
    D_hat = np.diag(A_hat.sum(axis=1))
    D_inv_sqrt = np.diag(1.0 / np.sqrt(np.diag(D_hat)))
    return D_inv_sqrt @ A_hat @ D_inv_sqrt


# Compute normalized adjacency
A_norm = normalize_adjacency(A)

print("Normalized adjacency matrix (first 4x4 block):")
print(np.round(A_norm[:4, :4], 3))
print("\nNotice: diagonal is non-zero (self-loops) and rows are normalized.")

# Test a single layer
layer = GCNLayer(3, 4)
H_out = layer.forward(X, A_norm)
print(f"\nInput features:  {X.shape}")
print(f"Output features: {H_out.shape}")
print("Each node now has features informed by its NEIGHBORS!")
```

Part 3: Full GCN Network — Stack Layers for Classification

In [3]

```python
# ============================================================
# Full GCN for node classification
# ============================================================

class GCN:
    """2-layer Graph Convolutional Network."""

    def __init__(self, in_features, hidden_features, out_classes):
        self.layer1 = GCNLayer(in_features, hidden_features)
        self.layer2 = GCNLayer(hidden_features, out_classes)

    def softmax(self, z):
        exp_z = np.exp(z - z.max(axis=1, keepdims=True))
        return exp_z / exp_z.sum(axis=1, keepdims=True)

    def forward(self, X, A_norm):
        self.H1 = self.layer1.forward(X, A_norm)
        self.Z2 = self.layer2.forward(self.H1, A_norm)
        # Softmax for classification (override ReLU on last layer)
        self.Z2_pre = self.layer2.Z  # Pre-activation
        self.probs = self.softmax(self.Z2_pre)
        return self.probs

    def loss(self, probs, y):
        """Cross-entropy loss."""
        N = len(y)
        log_probs = -np.log(probs[np.arange(N), y] + 1e-8)
        return np.mean(log_probs)

    def backward(self, y):
        """Backprop through the full network."""
        N = len(y)

        # Gradient of cross-entropy + softmax
        dZ2 = self.probs.copy()
        dZ2[np.arange(N), y] -= 1
        dZ2 /= N

        # Override the ReLU gradient for the output layer
        # (we used softmax, not ReLU, on the output)
        self.layer2.dW = (self.layer2.AH.T @ dZ2) / N
        self.layer2.db = np.mean(dZ2, axis=0)
        dAH = dZ2 @ self.layer2.W.T
        dH1 = self.layer2.A_hat.T @ dAH

        # Backprop through layer 1
        self.layer1.backward(dH1)

    def update(self, lr):
        for layer in [self.layer1, self.layer2]:
            layer.W -= lr * layer.dW
            layer.b -= lr * layer.db

    def train(self, X, A_norm, y, lr=0.01, epochs=200):
        losses = []
        accs = []
        for epoch in range(epochs):
            probs = self.forward(X, A_norm)
```

```
            l = self.loss(probs, y)
            losses.append(l)

            preds = np.argmax(probs, axis=1)
            acc = np.mean(preds == y)
            accs.append(acc)

            self.backward(y)
            self.update(lr)

            if epoch % 50 == 0:
                print(f"  Epoch {epoch:4d} | Loss: {l:.4f} | Acc: {acc:.3f}")
        return losses, accs


# Train the GCN!
gcn = GCN(in_features=3, hidden_features=8, out_classes=2)
print("Training GCN on graph classification...")
losses, accs = gcn.train(X, A_norm, y, lr=0.05, epochs=300)

# Final predictions
probs = gcn.forward(X, A_norm)
preds = np.argmax(probs, axis=1)
print(f"\nFinal predictions: {preds}")
print(f"True labels:       {y}")
print(f"Accuracy:          {np.mean(preds == y):.1%}")
```

In [4]

```python
# Visualize training and learned representations
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Loss curve
axes[0].plot(losses, color='steelblue')
axes[0].set_title('Training Loss')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Cross-Entropy')
axes[0].grid(True, alpha=0.2)

# Accuracy
axes[1].plot(accs, color='coral')
axes[1].set_title('Classification Accuracy')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].set_ylim(0, 1.05)
axes[1].grid(True, alpha=0.2)

# Learned node embeddings (hidden layer output)
H_hidden = gcn.layer1.forward(X, A_norm)
if H_hidden.shape[1] >= 2:
    for cls in [0, 1]:
        mask = y == cls
        label = f'Class {cls}'
        color = 'steelblue' if cls == 0 else 'coral'
        axes[2].scatter(H_hidden[mask, 0], H_hidden[mask, 1],
                        c=color, s=200, label=label, edgecolors='black', zorder=5)
        for i in np.where(mask)[0]:
            axes[2].annotate(str(i), (H_hidden[i, 0], H_hidden[i, 1]),
                             ha='center', va='center', fontsize=9, fontweight='bold', color='white')

axes[2].set_title('Learned Node Embeddings (Layer 1)')
axes[2].legend()
axes[2].grid(True, alpha=0.2)

plt.suptitle('GCN FROM SCRATCH: Node Classification', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('w4_01_gcn_results.png', dpi=100, bbox_inches='tight')
plt.show()

print("\nKEY INSIGHT: After message passing, connected nodes of the same class")
print("have SIMILAR embeddings. That is what GNNs learn: structural similarity.")
```

Part 4: Why Message Passing Works — Intuition

Each GCN layer does ONE round of "talk to your neighbors":

- **Layer 1:** Each node sees its immediate neighbors (1-hop)
- **Layer 2:** Each node sees neighbors-of-neighbors (2-hop)
- **Layer K:** Each node has a K-hop receptive field

This is the graph equivalent of a CNN's receptive field growing with depth.

Key Differences from MLPs:

	MLP	GCN
Input	Fixed-size vector	Variable-size graph
Sharing	No weight sharing	Same W for all nodes

Structure	Ignores structure	Exploits connectivity
Inductive bias	None	Homophily (connected nodes are similar)

■ Self-Check

- [ ] You can explain message passing in one sentence: "Each node aggregates its neighbors' features, transforms, and activates"
- [ ] You understand why A + I (self-loops) is needed: a node should retain its OWN features
- [ ] You can draw how 2 layers give 2-hop receptive field
- [ ] Your GCN achieves high accuracy on the toy graph

➡■ Next: `W4_02_PyTorch_GNN.ipynb` — Real graphs with PyTorch Geometric